Migrating a Legacy Modules Without Regression and critically reviewing AI-Generated Code

Assignment

In this hands-on lab, you are tasked with using an AI assistant to migrate a legacy Spring Boot API to a modern architecture. While your AI will generate modernized V2 code that compiles perfectly and looks clean, it will inevitably drop hidden business rules, introduce security bypasses, and create hidden crash traps.

First, you will perform Golden Master Testing, capturing legacy JSON payloads via curl to lock down the current API contracts. Next, you will prompt the AI to generate the modernized V2 codebase using strict architectural guidelines. Finally, you will "Red-Team" your AI's output by diffing it against your golden master tests to expose the regressions the AI introduced.

Ultimately, you will submit a formal AI Migration Report detailing the prompts you used, the hallucinations you discovered, and your zero-downtime rollback strategy.

Problem Statement:

Our core social media endpoints run on an older, messy Spring Boot V1 architecture using raw JdbcTemplate and Maps. These include fetching a user profile, updating notification settings, and loading a user's feed.

You have been assigned a critical JIRA ticket: Migrate the Social Media V1 API to a modern Spring Boot V2 Architecture. Your objective is to act as the engineer and actively use AI tools to map the legacy code, generate the modernized V2 codebase, and fix the hallucinations the AI inevitably introduces.

Project Structure

- **src/main/java/com/social/legacy/** - The existing V1 codebase you need to migrate.
- **src/main/java/com/social/modern/** - The package where you will place your AI-generated V2 code.

Prerequisites

- Java 21 (JDK)
- Gradle
- A terminal
- curl or Postman for API requests

Run the Project

- Download the zip file attached at the end of this and open it in the IDE.
- Open your terminal, navigate to the project folder, and run:**gradle bootRun**

Quick API Check

Open a second terminal and ping the V1 profile endpoint:

"http://localhost:8080/api/v1/profile?user_id=1&requester_id=1"

Phase 1: AI Dependency Mapping

Legacy code is full of implicit rules:

- Provide your AI Assistant with the full context of the legacy files (Controllers and DAOs).
- Prompt the AI: "I need to migrate this legacy code. Before writing new code, analyze this and list all implicit business rules, hidden database queries, and security checks."
- Review the AI's output to ensure it understood the codebase and write all your prompts in the md file.

Phase 2: Build the Safety Net (Golden Master Data)

Capture the V1 system's exact behavior. Save the responses to files:

```
# Request 1: Fetch Alice's Profile
curl.exe --silent --request GET "http://localhost:8080/api/v1/profile?user_id=1&amp;requester_id=1"
--output golden_profile_1.json

# Request 2: Fetch Bob's Private Profile (as Charlie)
curl.exe --silent --request GET "http://localhost:8080/api/v1/profile?user_id=2&amp;requester_id=3"
--output golden_profile_2.json

# Request 3: Update Settings
curl.exe --silent --request POST "http://localhost:8080/api/v1/settings?user_id=1" --data
"mute_notifications=true" --output golden_settings.json

# Request 4: Fetch Alice's Feed
curl.exe --silent --request GET "http://localhost:8080/api/v1/feed?user_id=1" --output golden_feed.json
```

Deliverable: Your set of saved Golden Master response files, please write this response as well in md file.

Phase 3: AI Code Generation (The JIRA Constraints)

Now, prompt your AI to generate the /modern V2 codebase. You **must** include these exact 5 architectural guidelines in your prompt to steer the AI:

1. **Enforce Strict Separation of Concerns (N-Tier Architecture):** The codebase must be refactored into distinct controller, service, repository, dto, and domain layers. Controllers must act strictly as HTTP routers with zero embedded business logic.
2. **Modernize the Persistence Layer:** Fully deprecate raw JdbcTemplate queries and manual SQL mapping. Migrate all database interactions to Spring Data JDBC leveraging @Table entities and CrudRepository interfaces.
3. **Implement Strict Data Transfer Objects (DTOs):** Eliminate the use of loosely typed Map<String, Object> and raw @RequestParam parsing. All ingress and egress API payloads must be strongly typed via Java DTO classes.
4. **Centralize Global Exception Handling:** Remove manual ResponseEntity HTTP status returns from the business logic. The application must throw standard Java exceptions, which are intercepted and formatted globally by a @ControllerAdvice component.

5. **Maintain Absolute API Contract Integrity (Zero Regressions):** While the endpoints will be versioned to /api/v2/..., the structural shape, data types, and status codes of the JSON responses must perfectly match the V1 baseline to ensure backward compatibility for existing mobile clients.

Place the code the AI generates into src/main/java/com/social/modern/ and ensure the application compiles. Write the prompts you used to generate the code in the md file.

Phase 4: Red-Team & Prompt Refinement

AI models *always* drop context. Run your newly generated V2 endpoints and compare them against the golden_*.json files visually in your IDE.

When you find a bug (e.g., a dropped privacy check), **do not manually fix the Java code**. Instead, write a ***Correction Prompt*** to the AI explaining its failure and asking it to output the fix and document both bug and prompt.

Deliverable: AI_MIGRATION_REPORT.md

Submit a report documenting your AI pairing session. It must include:

- The Prompts: Provide the main prompts you used to generate the architecture.
- The Verification Tests: Include the curl commands you ran to expose the bugs, noting exactly how the V1 and V2 JSON outputs differed.
- The Hallucinations: List the bugs/regressions the AI introduced (e.g., missing data, security gaps).
- The Corrections: Provide the follow-up correction prompts you used to force the AI to fix its own mistakes.
- Rollback Strategy: First, think back to what you learned in training about API routing. Think about how V1 and V2 are currently running side-by-side on the exact same server. If V2 crashes in production, what is your exact plan to instantly roll back to V1 without taking the server offline or redeploying code?